MIDDLESEX Community College

Tools and Technologies for Tech Writers 2026

Week 11

Sample Bookmap

Middlesex Community College 2026

This document was prepared as an assignment for the Middlesex Community College Tools and Technologies for Technical Writers class, Winter semester 2026.

Prepared by Millie Driz.

The name is defined by the value of the author key defined in your bookmap. I set them up to be your name.

Contents

My First Topic

This is a sample topic. It is in your homework folder (Millie) and is not shared.

This is a paragraph.

Example Topic

The topic is the default, unspecialised topic type of DITA.

The `<topic>` is the basic building block of DITA. All other topic types are derived/specialized from the topic.

In general, when authoring, avoid using `<topic>` type topics because they carry no special meaning. However, they do serve some very important purposes:

- Provide the basic topic type to specialize from.

  If you choose to create your own topic types, you can start with a topic and go from there.

- Provide a default for conversion work.

  When converting to DITA, you don't always do all the work you should do to identify topic types. You can convert everything to topics and then sort them out later.

- Great for what I call "container topics"

  Most DITA generation tools allow for auto-generated links, especially child links. To structure your content, you can create a topic with nothing but a title. The child links (and optional short descriptions) can be included automatically.

Topic Building Blocks

Brief rundown of the elements you can use in `<topic>` type topics.

Structure Blocks

These are the elements that make up a topic.

topic	The defining container
title	All topics need a title. Note that there is only one `<title>` element. All things that can use a title use this same element, whether that is a topic, section, figure, table, etc.
shortdesc	The short description for the topic. This is the first sentence of the topic. This element can be picked up in search results or when generating links. Alternatively, you can use an `<abstract>`. The `<abstract>` allows for more content than a `<shortdesc>`, but it is not extracted and reused the same way. However, you can place multiple `<shortdesc>` inside an `<abstract>`, which can be useful when dealing with conditional content.
prolog	A place to put all the metadata for the topic. In general, nothing in this section emits in the topic. This can include: • Author • Source

- Publisher
- Copyright
- Change History List (technically a specialization, but I'm listing all the prolog things here)
- Critical Dates
- Metadata: keywords, index terms, and 'other meta' such as any name value pair you need.

body The container that holds the content of the topic

related-links A list of links to other topics of interest (instead of having links in the text).

Body blocks

These are the elements that can be used as blocks inside of the `<body>`.

Technically, the `<topic>` should not contain specializations. However, the technical content components are usually just included with.

bodydiv Generic container to group content. It can only be used inside of a `<body>`, so it's mainly for using for specialization.

codeblock This is a specialization from the programming domain, but it indicates multiple lines of code.

div A generic container. It can group content with no semantic meaning. In general, avoid using, except for when grouping together a set of blocks you want to reuse instead of dealing with using the fancy conref attributes.

dl Definition list. You can provide a term and at least one definition. Super useful. Can be displayed as a funky formatted list or as a table. If you add an `@id` attribute to the `<dlentry>` element, the `<dt>` is automagically returned in the `<xref>`.

data Basic name value pair for specialization. Does not emit by default. In general, don't use except when you want to support funky processing without a specialization (if you know how to change the formatting).

data-about Funky extension thing used with `<data>`. Used for specializations. Don't use.

draft-comment One of my favorite elements. The DITA-OT includes a parameter that excludes this element. Enables you to include comments and questions in your writing with less fear that it will show up in the output.

equation-block To define where you're inserting an equation using `<mathml>` or just an image. Equations often need special formatting or funky fonts, so identifying them can be useful.

equation-figure As `<equation-block>`, but you can add a `<title>`.

example A specialized section that identifies the content as an example.

fig A grouping that identifies a *figure*. Generally, this is an image with a caption (`<title>`), but could contain text or a codeblock.

foreign	A special element for identifying things that aren't DITA. Available for specialization. Do not use.
hazardstatement	Fancy specialized Hazard Statements for https://blog.ansi.org/ansi/ansi-z535-1-2022-safety-colors-standard/ and https://www.iso.org/standard/51021.html.
image	How you link to an image.
imagemap	A special image where you can identify zones in the image and associate links with those zones.
lines	A block where white space is preserved, but not necessarily using monospaced font. Often used for addresses or poetry.
lq	Long quote. When you want to identify a quotation that contains multiple paragraphs.
mathml	An element that can contain MATHML, an XML based language for rendering equations. https://developer.mozilla.org/en-US/docs/Web/MathML
msgblock	A special block for messages, usually from software.
note	A container for short(ish) content you want to emphasize in some way. The default DITA contains several types of notes, including tip, warning, caution, and important.
object	An element to reference something not-DITA, such as a video.
ol	Ordered list, just like HTML.
p	A paragraph, just like HTML.
parml	A parameter list. A special list, like `<dl>`, but for parameters. Not used very often.
pre	Preformatted text, just like HTML.
required-cleanup	Special element that you should only see during conversion work. If a conversion transform finds a lump of content it can't deal with, wrap it in `<required-cleanup>` and let a human deal with it.
screen	A specialization to identify content you see on a screen. Very useful when dealing with command line content.
section	A division to group content with an optional title. In general, section titles are not included in the table of contents.
simpletable	A basic table. In DITA 1.3, does not allow row or column spanning. You can only control whether or not separators display.
sl	Simple list. Provides a list without bullets or numbering.
sort-as	Special element meant to be used with indexes.
svg-container	Special element if you want to include raw SVG code in your topic.
syntaxdiagram	Super specialized element for syntax diagrams, used in some programming languages.
table	A CALS based table.

ul	Unordered list. Same as HTML.
unknown	Element for specialization. Do not use.

Inline elements

There are many block elements that can be used inside of other block elements. For example, you can have an `<image>` or `<ul>` inside of a `<p>`. I am not including those elements again here.

I personally prefer keeping my elements separate. I do not like to have a list inside of a `<p>`, I prefer the list to be after the paragraph. This is my best practice. However other people prefer having the block items inside of paragraphs so that it's easier to reuse them together. That may have made more sense in earlier editions of DITA, but now that you can define conrefs to contain multiple elements, or use a `<div>` to combine them, I prefer to keep things as tidy as possible. Since a lot of formatting is either CSS or XSL-FO based, inheritance matters. A `<ul>` inside of a `<p>` may inherit different margins etc. that just makes troubleshooting formatting issues a pain.

Generic Elements

b	Generic bold. Avoid if possible and pick something semantic.
line-through	Formatting to strike through content.
markupname	Generic identifier of some sort of mark up. Meant for specialization.
overline	Formatting to have a line over the text.
sub	Subscript - Use as needed.
sup	Superscript - Use as needed.
tt	Generic monospace formatting. Same as HTML. Try to use something semantic.
u	Generic underline formatting. Same as HTML. Try to use something semantic.

Semantic Elements

abbreviated-form	If you use all the special bits of glossaries and the abbreviation domain, use this to identify the abbreviated form of a term.
apiname	For API names
cite	To identify a citation.
cmdname	A command name.
codeph	A code phrase
equation-inline	Like `<equation-block>`, but you can have your math inline.
filepath	For paths when describing things on computers.
fn	Footnote. You enter it right in the text, and the processing will put it at the bottom of the page.
indexterm	If you need to have index terms in your content, you can put them where needed, not just in the `<prolog>`

indextermref	If you need to do any of that "see also" stuff in your index.
keyword	Identifies an important word. Generally has no formatting, but can help with search results if your processing supports it. Also useful for rendering keys of simple phrases.
menucascade	In the software world, you constantly need to identify a set of nested options in menus. Use the `<menucascade>` to identify and format them for you. For example: Select **File** > **Open**.
msgnum	If your system provides codes or IDs for system messages, use this to identify it.
msgph	For inline system messages.
numcharref object	XML Domain to identify when you reference funky characters by Hex codes. For example # is `&#x38f;`.
option	For software or programming, an option.
parameterentity	XML Domain for parameter entities in schemas.
parmname	For software or programming, a parameter name.
ph	A phrase. No semantic meaning or formatting, but crazy useful for inserting keys or marking sentences for conditions.
q	An inline quote.
synph	Syntax phrase. Similar to codeph, but slightly different. Sorta like syntax is an algebra equation a + b = c but codeph is a math equation 1 + 2 = 3.
systemoutput	Response from a system.
term	Identifies something as a word of importance. You can hook it up with keys to a glossary entry to link to the definition. For example, *glossterm*.
textentity	XML domain to identify text entities in schemas.
tm	Use to identify something as a trademark. For example Soon™.
uicontrol	Something in the user interface.
userinput	User input, i.e. something you type in.
varname	Variable name
wintitle	Window title. Can be used for a window, screen, dialog box, etc.
xmlatt	XML domain attribute name
xmlelement	XML Domain element name
xmlnsname	XML Domain name space
xmlpi	XML Domain processing instruction
xref	How you define links.

Specialization Elements

boolean	Meant for specialization, an element that can be true or false. Do not use.

state	A name value pair for specialization. Do not use.
text	Mainly intended as a specialization element. Avoid using.

Example Concept Topic

An example DITA topic of type concept. This is a shared topic.

This is a topic of type concept.

Concept topics are very similar to topic type topics. A `<concept>` does not have a lot of structure to it.

Concepts are intended for more free-form text, with illustrations, etc.

Sections

Concepts can have sections. However, once you start a `<section>`, you cannot return to the main body of the `<concept>`.

While sections may seem like just adding a sub-heading, by default, section titles are not included in the table of contents. They can be useful for grouping concepts, highlighting ideas, etc. If you add an `@id` attribute to the `<section>` element, you can link to it, and the title of the section is automatically pulled in by the `<xref>`.

Section Specialization: Example

An example is just a fancy section. However, it does add the semantic idea of "this is an example" around the contents.

Example Task Topic

A DITA topic of type task. This is a shared topic. Task topics are the most structured.

This is the `<prereq>` where you list the prerequisites for this task. This section is optional.

This is the `<context>` where you can provide a bit of background of things you should know before performing the task.

Some people consider this a "mini concept". If you realize you need a bit of lead in, but not enough for a stand alone topic, use the `<context>` in the task topic instead of a stand alone concept topic. This section is optional.

1. This is a `<step>` in a task topic. It is part of the `<steps>` element. You can use `<steps>` for numbered steps or `<steps-unordered>` for a non-numbered (usually bulleted) steps.
2. These are the heart and soul of task topic.

   There are many additional special elements that can be used in a step to provide more information.
3. Both the `<steps>` and `<steps-unordered>` elements are optional.

That's correct, you can have a task with no procedure.

This is the `<result>` which lets you explain what happens after you complete the steps. This element is optional.

This is `<tasktroubleshooting>` where you can provide troubleshooting information for the entire task.

Not to be confused with `<steptroubleshooting>` which can be used for troubleshooting information for specific steps.

This element is optional.

This is an `<example>` you can use to provide an example for the entire task. This is not to be confused with the `<stepxmp>` which can provide a quick example for a specific step.

This element is optional.

This is the `<postreq>` where you describe what to do after completing the procedure.

This element is optional.

That's correct, all of these elements are optional. It is possible to have a valid task topic with only a `<title>` and `<shortdesc>`. However, the order of these main elements is not optional. For example, you cannot switch the order of the `<context>` and `<prereq>` without a specialization.

Example Reference Topic

Example Troubleshooting

Condition

This is the condition. What is the problem, or at least the symptoms.

Cause

What is the root cause of the issue.

Remedy

Who should fix this (Admin, tech, user)

1. This can be a procedure on how to fix the thing.
2. Yes, this is the same step as in a task.

Example Glossary

glossterm

This is the `<glossdef>` where you can write the definition.